A SYNOPSIS ON

PRAMANA

Citation-Verified, Fully Local AI Legal Research Assistant for Indian Criminal Law across the IPC–BNS Transition

Submitted in partial fulfilment of the requirements for the award of the degree of

Bachelor of Technology

in _____

Submitted by

Suvan Sharma

University Roll No.: _____

Team: The Spartan Coders

Under the guidance of

Aditya Sharma

Project Mentor

Department of _____

I.K. GUJRAL PUNJAB TECHNICAL UNIVERSITY

Punjab, India

October 2026

CONTENTS

Abstract 2
1. Introduction 3
2. Problem Statement and Motivation 3
2.1 Problem statement 3
2.2 Motivation 4
3. Objectives 4
4. Literature Survey 5
4.1 Research gap 6
5. Feasibility Study 6
5.1 Technical feasibility 6
5.2 Economic feasibility 6
5.3 Operational feasibility 6
6. Proposed System and Methodology 7
6.1 System architecture 7
6.2 Modules 7
6.3 Dataset 9
6.4 Development methodology 9
7. Hardware and Software Requirements 10
8. Testing, Evaluation and Results 10
8.1 Citation verification 10
8.2 Retrieval 11
8.3 End-to-end statute identification 11
8.4 IL-TUR benchmark 11
8.5 Worked example 12
9. Snapshots of the System 12
10. Applications, Limitations and Future Scope 14
10.1 Applications 14
10.2 Limitations 14
10.3 Future scope 14
11. Project Plan 15
12. Conclusion 15
References 16

ABSTRACT

On 1 July 2024 India replaced the Indian Penal Code, 1860 (IPC) with the Bharatiya Nyaya Sanhita, 2023 (BNS). Almost every offence was renumbered, while offences committed before that date are still tried under the IPC, so lawyers must work with both codes for years. Large language models (LLMs) used for legal research are known to invent citations, and the transition makes this worse: the same section number denotes unrelated offences in the two codes (IPC 302 is murder, BNS 302 is wounding religious feelings; BNS 303 is theft).

Pramana is an AI-assisted legal research system for Indian criminal law built around one principle: *the language model proposes, deterministic code decides what is real*. Given case facts and an offence date, a deterministic router selects the applicable code; retrieval fuses statute similarity with votes from similar judgments; a small local LLM proposes candidate sections; and a section is admitted into the report only if it exists as a node of the applicable code in a citation graph built from the ingested statutes. Rejected proposals are fed back through a bounded retry loop. Later stages test each section against the facts, retrieve graph-linked precedents (bridging new BNS charges to their IPC predecessors through a reviewed 575-row mapping), draft counter-arguments that are re-verified after generation, and summarise appeal outcomes with pure SQL.

The system runs entirely on a laptop with a 4 GB GPU at zero recurring cost, so a client's facts never leave the machine. In every evaluation no non-existent or wrong-code section reached a report; with retrieval withheld the verifier caught 29% of the model's citations and every affected case recovered within two retries. Section choice remains the weak point: on the IL-TUR statute-identification benchmark the zero-shot system scores macro-F1 11.8 on the test split against 28.1 for the best supervised model, and this gap is reported openly.

Keywords: legal AI, retrieval-augmented generation, citation verification, hallucination, Bharatiya Nyaya Sanhita, Indian Penal Code, knowledge graph.

1. INTRODUCTION

Legal research is the first step of every criminal matter: a lawyer reads the facts, identifies the offences they disclose, finds the statutory text and locates precedents in which courts applied those sections. In India this work is done with commercial databases or by hand, and it has become harder since the three new criminal laws came into force on 1 July 2024. The BNS re-enacts most IPC offences under new numbers, merges the IPC's separate definition and punishment sections (IPC 378 defines theft and 379 punishes it; BNS 303 does both), moves some provisions into sub-sections (common intention, IPC 34, is now BNS 3(5)) and drops others (adultery, IPC 497).

Generative AI promises faster research, but general-purpose chatbots and even commercial legal tools have been shown to produce confident, fabricated citations [2], [3]. During development the 4-billion-parameter model used in this project was asked what BNS 303 covers; it described a bomb-threat offence. A lawyer who relies on such an answer cites a section that does not say what the brief claims.

Pramana addresses this specific, checkable failure structurally rather than statistically. The applicable code is chosen by the offence date, never by the model; every section the model proposes is looked up in a citation graph built from the actual statutory text of that code; and mechanical legal conditions (five persons for an unlawful assembly, a death for homicide) are enforced by rules. The LLM is used only where language understanding is unavoidable, and each of its outputs is checked again by code. The whole system runs offline on commodity hardware, which keeps client facts confidential and costs nothing to operate.

2. PROBLEM STATEMENT AND MOTIVATION

2.1 Problem statement

To design and build a legal research assistant that, given the facts of a criminal case and the date of the offence, identifies the applicable sections of the correct code (IPC or BNS), **guarantees that every cited section exists in that code**, retrieves relevant precedent judgments, generates likely defence counter-arguments and reports how appeals on those sections were decided — running entirely on a local laptop at zero recurring cost.

2.2 Motivation

- **Dual-code period.** Offences before 1 July 2024 stay under the IPC, later ones fall under the BNS. Every lawyer must handle both codes, and mixing them produces citations that look right but are wrong.
- **Hallucinated citations.** LLMs fabricate or misattribute legal citations [2], [3]; in law a wrong citation is not a cosmetic error.
- **Scarce BNS case law.** Very few cases charged under the BNS have reached the Supreme Court; useful precedent for a new-code offence lies under its old IPC number.
- **Confidentiality.** A lawyer types a client's real facts. Sending them to a cloud AI service that may retain or train on prompts is unacceptable, so generation must be local.
- **Cost and access.** Commercial legal databases are expensive. A free, offline tool helps students, legal-aid clinics and small practices.

3. OBJECTIVES

The objectives of the project are:

1. To build a deterministic **offence-date router** that selects the IPC for offences before 1 July 2024 and the BNS for offences on or after it.
2. To ingest, clean and chunk the IPC, the BNS and Supreme Court and High Court judgments into a searchable vector store, with metadata (source, court, date, sections involved) on every chunk.
3. To build a **citation graph** of sections and judgments and verify every proposed citation against it, with a feedback retry loop for rejected proposals.
4. To identify the applicable sections from free-text facts using retrieval-augmented generation with a small local LLM, and to test each verified section against the facts.
5. To retrieve precedents for each charge, bridging BNS sections to their IPC predecessors through a reviewed IPC–BNS mapping table.
6. To generate grounded defence counter-arguments and section-level outcome analytics, with an explicit "insufficient data" response when retrieval finds nothing relevant.

7. To deliver the system through a FastAPI backend and a web interface in which verified, doubted and rejected sections are visually distinct.
8. To evaluate the system on hand-written, real court-order and published benchmark (IL-TUR) data and report the results honestly, including negative results.

4. LITERATURE SURVEY

The literature relevant to this project falls into four areas: Indian legal NLP benchmarks, legal embedding models, retrieval-augmented generation, and hallucination in legal AI. Table 4.1 summarises the works that most directly shaped the design.

Table 4.1: Summary of related work

Work	Approach	Finding / relevance
IL-TUR, Joshi et al., ACL 2024 [4]	Benchmark of Indian legal tasks incl. Legal Statute Identification (LSI, 100 IPC labels)	Best supervised LSI macro-F1 28.1 (LeSICiN). Used here for external evaluation.
LeSICiN, Paul et al., AAAI 2022 [5]	Heterogeneous graph of facts, sections and citations; supervised	Graph structure helps statute identification; needs 42K training documents.
InLegalBERT, Paul et al., ICAIL 2023 [6]	BERT pre-trained on Indian legal text	Strong for supervised tasks; its sentence variant was the planned embedder.
Premasiri et al., RANLP 2025 [9]	General vs legal-domain embedders for prior case retrieval	General LLM-based embedders match or beat legal ones — confirmed by this project's own bake-off.
NyayaAnumana, Nigam et al., COLING 2025 [7]	Largest Indian judgment-prediction corpus	Source of the curated High Court subset (1,827 judgments).
ILSIC, Paul et al., EACL 2026 [8]	Statute identification from layperson queries; zero-shot LLMs	Zero-shot GPT-4.1 reached 14.4 macro-F1; retrieval is the bottleneck.
RAG, Lewis et al., NeurIPS 2020 [10]	Ground generation in retrieved documents	Basis of the generation layer (RAG, not fine-tuning).
Reciprocal Rank Fusion, Cormack et al., SIGIR 2009 [11]	Combine rankings by summing 1/(k + rank)	Used to fuse statute similarity with precedent votes.
Dahl et al., 2024 [2]; Magesh et al., 2024 [3]	Measured legal hallucination in chatbots and commercial RAG tools	Hallucinated citations persist even with RAG — motivates structural verification.

4.1 Research gap

Existing Indian legal systems are either supervised models trained for one task on the IPC only, or general chatbots with no guarantee that a citation exists. None (i) handles both codes with deterministic routing, (ii) verifies every citation against a code-scoped graph before display, (iii) bridges new-code charges to old-code precedent, and (iv) runs fully offline on a laptop. Pramana is designed to fill this gap. A domain fine-tuned Indian legal model (AALAP) has also been reported to score below an untouched base model on the AIBE benchmark while a retrieval-augmented approach scored highest, which is why this project uses RAG rather than fine-tuning.

5. FEASIBILITY STUDY

5.1 Technical feasibility

All components are mature open-source tools: Python, PyTorch (CUDA), sentence-transformers, ChromaDB, NetworkX, LangGraph, FastAPI and Ollama. The main technical risk was memory: the embedding model (about 2 GB at fp16) and the 4-bit LLM (about 3.7 GB loaded) cannot share a 4 GB GPU. This was solved with a per-case **VRAM handoff** — the LLM is evicted, the embedder encodes the query and pre-fetches the judgments needed later, and is then freed before generation.

5.2 Economic feasibility

The project has **zero recurring cost**. No paid API, cloud service or licence is used; all datasets are public or free for research, and the models are open-weight. The only cost is the existing laptop. The total disk budget was held under 20 GB by curating the data (English-only judgments, a filtered subset of NyayaAnumana, evaluation splits only of IL-TUR).

5.3 Operational feasibility

The system starts with one double-click (run_web.bat), works without an internet connection, and needs no account or API key. A live analysis takes about 1–3 minutes on the development laptop, and saved sample matters open instantly. The interface reads the offence date from the facts and asks for it when it is missing or ambiguous.

6. PROPOSED SYSTEM AND METHODOLOGY

6.1 System architecture

The system has an offline part, built once, and a runtime pipeline executed per case (Figure 6.1). The runtime pipeline is a LangGraph state machine with a shared state (case_facts, offence_date, candidate_sections, verified_sections, precedents, analytics, defenses, retry_count).

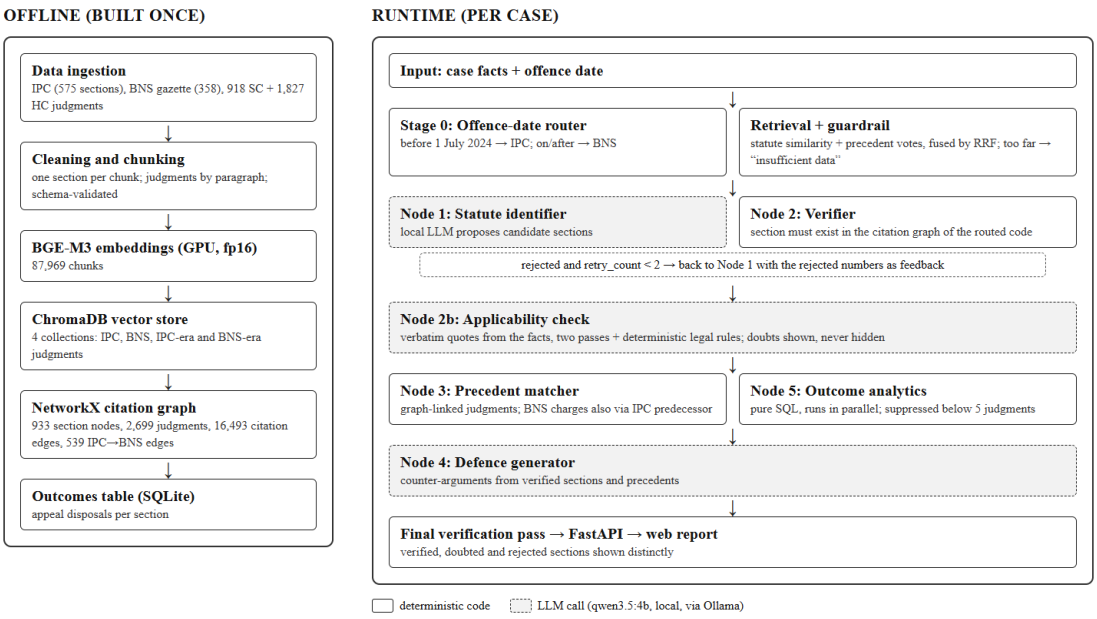


Figure 6.1: System architecture of Pramana

6.2 Modules

- **Offence-date router (Stage 0).** A plain function, not an LLM: a date before 2024-07-01 selects the IPC collections, any later date the BNS collections. Future and ambiguous dates are rejected (08/09/2024 is a different day in India and the US). An offence period that straddles the date is analysed under both codes with a warning.
- **Intake.** Real inputs are often court orders dominated by procedure. A deterministic test detects an order; one LLM call then restates only the alleged facts, and the restatement is kept only if 60% of its words occur in the original.
- **Retrieval.** Facts are encoded with BGE-M3 and matched against statute text. Similar judgments also *vote* for the sections they applied (an IPC judgment votes in a BNS case only through the verified mapping). The two rankings are fused by Reciprocal Rank Fusion: $RRF(x) = 1/(k + r_stat(x)) + 1/(k + r_vote(x))$, with $k = 60$.

- **Insufficient-data guardrail.** If the closest judgment is too far from the facts (cosine similarity below 0.576, calibrated on criminal and non-legal inputs), the LLM is not called and the nearest sections are returned for manual review.
- **Node 1 – Statute identifier.** The local LLM (qwen3.5:4b) reads the facts and about 14 retrieved candidates and proposes sections with JSON-schema-constrained output, temperature 0 and a fixed seed. It first records a fact profile (number of accused, shared plan, injury grade, weapon, intent).
- **Node 2 – Verifier.** Each proposal is looked up with verify_section(graph, code, section). The code argument is mandatory, so "BNS 420" (no such section) and "IPC 302" in a BNS case are rejected. If any proposal is rejected and retry_count < 2, control returns to Node 1 with the rejected numbers listed as banned and a hint from the graph (e.g. "its BNS counterpart is 303").
- **Node 2b – Applicability check.** Existence is not applicability, so the LLM must quote the facts verbatim for each ingredient of each verified section, in two passes with the order reversed. Deterministic legal rules (headcount, joint liability, injury grade, death requirement, lesser-included offences, chargeability) override or supply judgments. Doubted sections stay in the report with the reason.
- **Node 3 – Precedent matcher.** Returns judgments that have a graph edge to each charge, ordered by similarity to the facts. For a BNS charge it also follows the mapping to the IPC predecessor: BNS 303 has no direct precedent in the corpus, while IPC 378 and 379 have 11 and 158.
- **Node 5 – Outcome analytics.** Runs in parallel with Node 3; pure SQL over a pre-extracted outcomes table, no LLM. Sections with fewer than five supporting judgments are suppressed, and figures are labelled as appeal disposals, not conviction rates.
- **Node 4 – Defence generator and final check.** Drafts up to four counter-arguments grounded in the verified sections and labelled precedents. A final deterministic pass checks that each argument targets a charged section, cites only supplied precedents that exist in the graph and quotes the facts verbatim.

6.3 Dataset

Table 6.1: Data used

Source	Content	Size after processing
IPC (civictech JSON + official 1997 text)	All sections, one chunk per section	575 sections
BNS (Gazette of India, Act 45 of 2023)	Parsed from the official gazette PDF	358 sections
Supreme Court judgments 2023–2026	English, criminal judgments applying a penal code	918 judgments
NyayaAnumana (High Courts)	Filtered to theft, hurt, cheating, breach of trust	1,827 judgments
IPC→BNS mapping table	Every row reviewed against gazette text	575 rows, 539 bridges
IL-TUR LSI (evaluation only)	Dev and test splits	100 documents each
Total indexed	Statute and judgment chunks in ChromaDB	87,969 chunks

The citation graph has 933 section nodes, 2,699 judgment nodes, 16,493 judgment-to-section citation edges and 539 IPC-to-BNS mapping edges. Data quality was checked by reading random samples, not only by schema validation; this found and fixed broken ligatures, running page headers, lost BNS titles and 7,035 CrPC citations wrongly stored as IPC sections.

6.4 Development methodology

The project followed an incremental, gate-driven model in seven phases (Section 11). Each phase had a written, measurable “done when” criterion and its own git branch, merged only when the criterion was met. Every method change was measured against the same code with only that change switched off (ablation switches), then scored once on a held-out set written before the change.

7. HARDWARE AND SOFTWARE REQUIREMENTS

Table 7.1: Hardware requirements

Component	Specification (development machine)
Processor	Multi-core x86-64 laptop CPU
GPU	NVIDIA RTX 3050 Ti, 4 GB VRAM (CUDA)
RAM	16 GB
Storage	About 20 GB free (budgeted ceiling)
Operating system	Windows 11

Table 7.2: Software and tools

Layer	Tool
Language	Python 3.12
Deep learning	PyTorch 2.6 (CUDA 12.4), sentence-transformers
Embedding model	BAAI/bge-m3 (1024-dimensional, fp16 on GPU)
Vector store	ChromaDB (embedded, four collections)
Citation graph	NetworkX
Orchestration	LangGraph
Language model	qwen3.5:4b (4-bit) served locally by Ollama
Backend / frontend	FastAPI; HTML/JavaScript web interface; Streamlit (alternative)
Storage of outcomes	SQLite
PDF processing	PyMuPDF
Testing / versioning	pytest (422 automated tests); Git

8. TESTING, EVALUATION AND RESULTS

Testing covered unit tests for every module (422 automated tests pass), a binary gate for each phase, and three kinds of evaluation data: hand-written fact patterns, real court orders from an external dataset, and the published IL-TUR benchmark.

8.1 Citation verification

- Adversarial set of 10 real and 10 fabricated section numbers (including BNS 420 and BNS 498A, real only in the IPC): **20/20 correct**.
- Offence-date routing: **30/30** runs correct (10 cases × 3 repetitions).
- Of 889 citations proposed across 263 saved evaluation runs, **0 unverified citations** were ever shown in a report.

Table 8.1: Hallucinations caught by the verifier

Condition	Proposals	Rejected	Leaked
30 real court orders, with retrieval	94	0 (0%)	0
15 cases, retrieval withheld (stress test)	68	20 (29%)	0

With retrieval withheld the model cited from memory, mostly old IPC numbers in BNS-era cases; all six affected cases recovered a verified section within two retries.

8.2 Retrieval

An embedding bake-off on 15 hand-written cases chose BGE-M3 over the planned InLegal-Sbert: recall@3 0.533 against 0.200. On the development set the candidate list read by Node 1 contained a relevant section in 26 of 27 cases.

8.3 End-to-end statute identification

Table 8.2: End-to-end results (a case is a hit if a correct section is charged)

Evaluation set	Hit rate	Precision
Development (15, hand-written)	15/15	35%
Held-out 2 (16 cases)	14/16	58%
Broad coverage (19 cases)	16/19	68%
Real court orders, fresh 30	13/30	21%

8.4 IL-TUR benchmark

Table 8.3: IL-TUR Legal Statute Identification, macro-F1 over 100 labels

System	Training	Macro-F1
LeSICiN [5]	Supervised	28.1
Hierarchical InLegalBERT	Supervised	26.2
Hierarchical LegalBERT	Supervised	21.7
Pramana, dev split (100 docs)	Zero-shot	9.8
Pramana, test split (100 docs, run once)	Zero-shot	11.8

The supervised models learn the benchmark's labels from 42,000 training documents, whereas Pramana is zero-shot with a 4-billion-parameter model on a laptop, so the gap is expected and is stated openly. Serious offences are handled well (test F1 for IPC 302, murder: 0.78); the losses come from minor companion offences that rarely reach the candidate list and from over-charging.

8.5 Worked example

Facts: a motorcycle parked outside the complainant's house at night is seen on CCTV being pushed away and ridden off by the accused; offence date 15 August 2024. The router selects the BNS; retrieval ranks BNS 303 (theft) first; Node 1 proposes BNS 303; Node 2 verifies it; Node 2b supports it with verbatim quotes; Node 3 returns IPC 379 theft precedents through the mapping; Node 4 argues that dishonest intention is not proved by movement alone. A live run takes about 70 seconds.

9. SNAPSHOTS OF THE SYSTEM

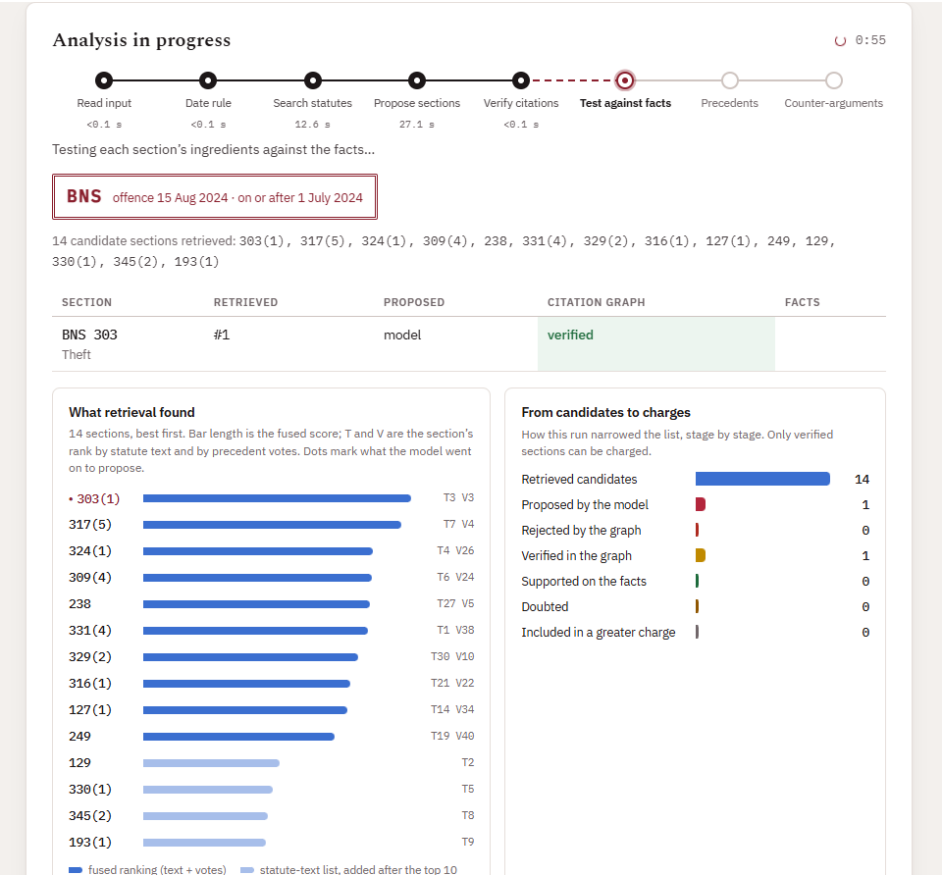


Figure 9.1: Live pipeline view during an analysis (retrieval scores and verification ledger)

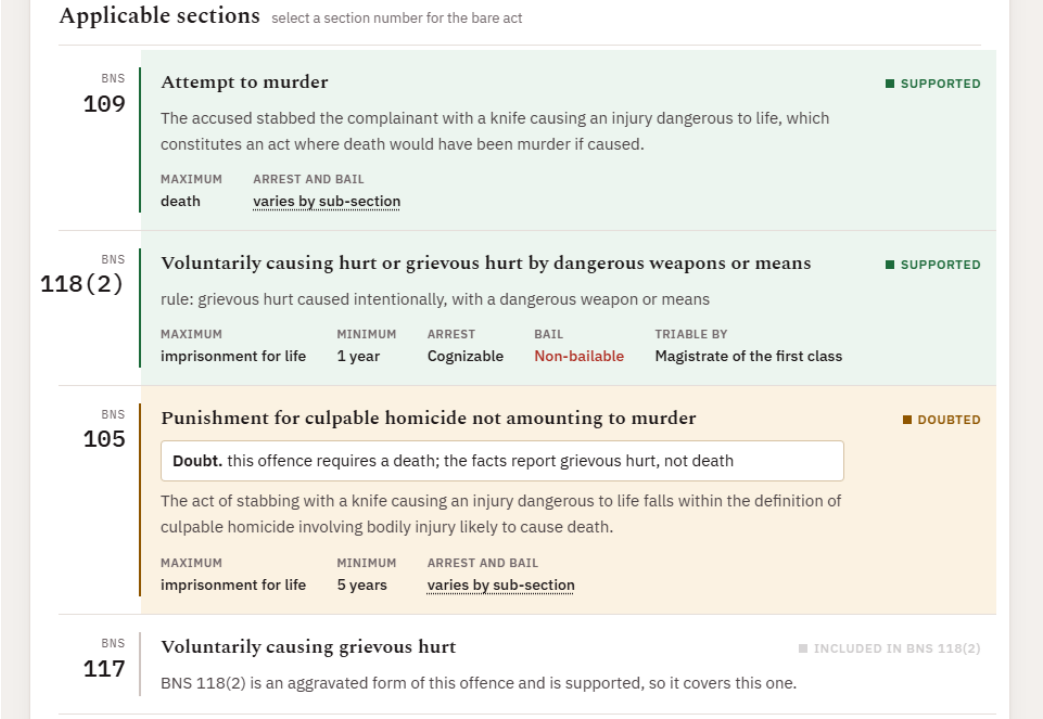


Figure 9.2: Report showing supported, doubted and included charges

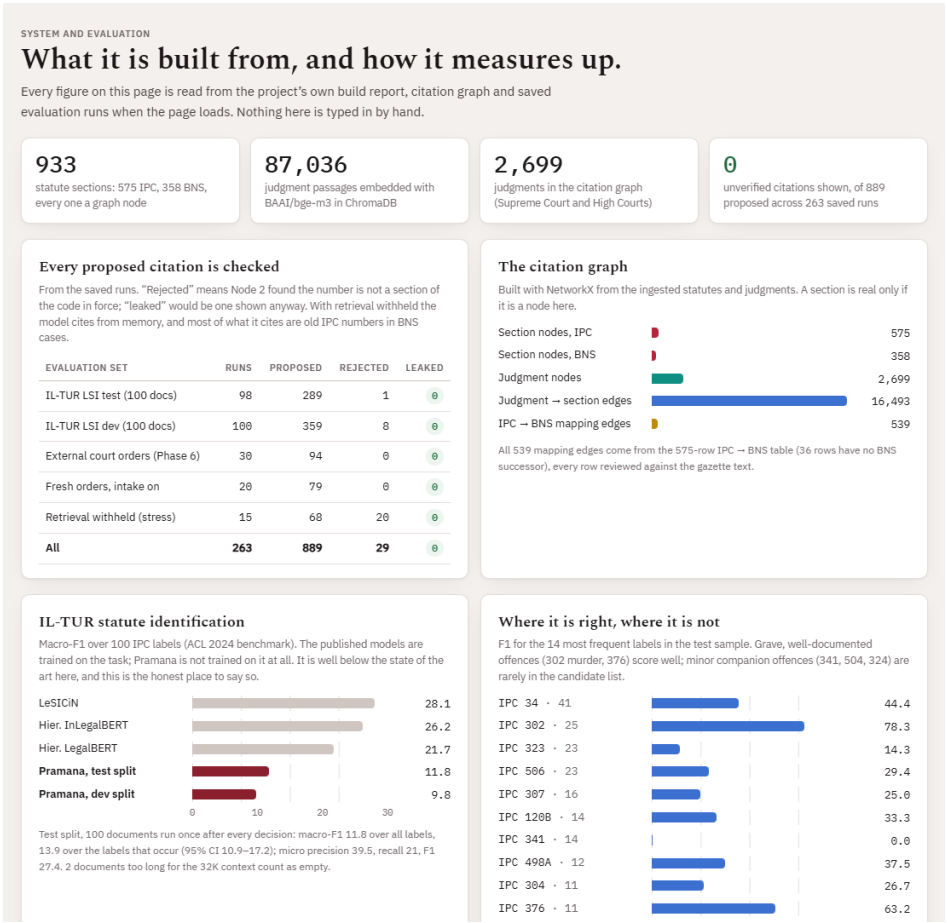


Figure 9.3: System and evaluation dashboard, read from the project's own records

10. APPLICATIONS, LIMITATIONS AND FUTURE SCOPE

10.1 Applications

- First-pass legal research for advocates and legal-aid clinics during the IPC–BNS transition.
- Teaching aid for law students comparing IPC and BNS provisions (section drawer and word-level IPC–BNS comparison).
- Confidential, offline case preparation where client facts must not leave the office.
- A reference design for trustworthy AI in other domains where citations must be real (medicine, regulation, taxation).

10.2 Limitations

- Precision of section choice is low and the system is below the supervised state of the art on IL-TUR.
- High Court data covers four offence families; BNS case law is still scarce, so post-2024 precedent comes mostly through IPC predecessors.
- Outcome analytics are appeal disposals, not conviction rates; no sentence ranges are reported.
- The IPC–BNS mapping and test keys were reviewed with AI assistance, not by a lawyer. Inputs must be English text; scanned documents are not read.

10.3 Future scope

- Lawyer review of the mapping table and evaluation keys.
- A precision model trained on (facts, section) pairs to reduce over-charging.
- Evaluation of precedent retrieval on IL-TUR Prior Case Retrieval, and broader High Court coverage.
- Support for the BNSS and BSA (the new procedure and evidence codes), Hindi input and OCR for scanned orders.

11. PROJECT PLAN

Table 11.1: Project phases

Phase	Work	Completion criterion	Status
1	Data layer: statutes, judgments, IPC–BNS mapping	Schema-valid data; 20 random chunks read and correct	Done
2	Retrieval: embeddings, ChromaDB, retrieve()	Relevant section in top 3 for 12 of 15 test cases	Done
3	Citation graph and verification	20/20 on real vs fabricated sections	Done
4	Router, guardrail, Nodes 1–2, retry loop	Routing 30/30; adversarial case survives retry	Done
5	Nodes 3, 4, 5 (precedents, arguments, analytics)	Worked example runs end to end	Done
6	Evaluation (external orders, IL-TUR)	Reproducible numbers in a results document	Done
7	FastAPI backend and web interface	Worked example in the browser, no manual steps	Done

12. CONCLUSION

Pramana shows that a zero-cost, fully local legal research assistant can make citation existence a guaranteed property rather than a hoped-for one, even across a wholesale renumbering of India's criminal code. It does so by routing deterministically, verifying every citation against a code-scoped citation graph, confining the language model to proposing and drafting, and checking each of its outputs again in code. The evaluation shows the strength (no invalid section ever reached a report) and the weakness (choosing the right section, where precision remains low) with real numbers. The project delivers a working system, a web interface, an evaluation record and a research paper.

REFERENCES

[1] Government of India, “The Bharatiya Nyaya Sanhita, 2023 (Act No. 45 of 2023),” Gazette of India, Ministry of Law and Justice, 2023.

[2] M. Dahl, V. Magesh, M. Suzgun and D. E. Ho, “Large legal fictions: Profiling legal hallucinations in large language models,” Journal of Legal Analysis, vol. 16, no. 1, 2024.

[3] V. Magesh, F. Surani, M. Dahl, M. Suzgun, C. D. Manning and D. E. Ho, “Hallucination-free? Assessing the reliability of leading AI legal research tools,” arXiv:2405.20362, 2024.

[4] A. Joshi, S. Paul, A. Sharma, P. Goyal, S. Ghosh and A. Modi, “IL-TUR: Benchmark for Indian legal text understanding and reasoning,” in Proc. ACL, 2024.

[5] S. Paul, P. Goyal and S. Ghosh, “LeSICiN: A heterogeneous graph-based approach for automatic legal statute identification from Indian legal documents,” in Proc. AAAI, 2022.

[6] S. Paul, A. Mandal, P. Goyal and S. Ghosh, “Pre-trained language models for the legal domain: A case study on Indian law,” in Proc. ICAIL, 2023.

[7] S. K. Nigam, B. D. Patnaik, S. Mishra, N. Shallum, K. Ghosh and A. Bhattacharya, “NyayaAnumana and INLegalLlama: The largest Indian legal judgment prediction dataset and specialized language model,” in Proc. COLING, 2025.

[8] S. Paul, R. Dogra, P. Goyal and S. Ghosh, “ILSIC: Corpora for identifying Indian legal statutes from queries by laypeople,” in Findings of EACL, 2026.

[9] D. Premasiri, T. Ranasinghe and R. Mitkov, “LLM-based embedders for prior case retrieval,” in Proc. RANLP, 2025.

[10] P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in Proc. NeurIPS, 2020.

[11] G. V. Cormack, C. L. A. Clarke and S. Büttcher, “Reciprocal rank fusion outperforms Condorcet and individual rank learning methods,” in Proc. SIGIR, 2009.

[12] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian and Z. Liu, “BGE M3-Embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation,” arXiv:2402.03216, 2024.

[13] Qwen Team, “Qwen3 technical report,” arXiv:2505.09388, 2025.

[14] LangChain, “LangGraph,” https://github.com/langchain-ai/langgraph, 2026.

[15] Ollama, https://ollama.com, 2026.